

World Wide Web enabled operational support systems — a testing perspective

R J Croucher

BT uses many operational support systems (OSS) in the course of its business. The major customer service system (MCSS) is one such OSS used for electronic dialogue with major customers. The client/server approach currently used is being superseded with a call for a World Wide Web (WWW) interface and adoption of a mid-tier server/thin-client approach. This paper examines how practical verification, validation and test (VV&T) techniques can be applied to such systems. Three WWW technologies are examined here as researched and implemented within the MCSS Bali project. The three technologies are Common Gateway Interface (CGI), Java and Citrix WinFrame.

1. Introduction

BT's major customer service system (MCSS) offers a service management function to both BT and customer users [1]. Its user interface has evolved over the years from a VAX VT100, through Unix X to its current PC Oracle Forms user interface. The current user interface requires a high specification PC, both in terms of random access memory (RAM) and central processing unit (CPU) power on the client desktop to run the application. The application is delivered using at least 6 floppy disks which expands to about 13 Megabytes in hard disk space.

A project team called 'Bali' was established at BT Laboratories in May 1996 to investigate and implement a WWW interface into MCSS providing for the first time a thin-client interface. The issue of managing the client desktop would then be replaced with the different management issue of maintaining the new mid-tier server and increased data traffic flow introduced. The mid-tier server can be considered as the '/' in the client/server computing paradigm (see Fig 1).

2. The VV&T techniques adopted on project Bali

2.1 Rapid application development (RAD)

The Bali project team was established as a virtual RAD-team using many of the principles identified within the dynamic systems development method (DSDM) and SI-Rapid [2], the latter being a Systems Integration (SI) technique of RAD taking into account the geographic spread of the people involved. Another excellent source of reference was section 3.11 of the VV&T Handbook [3]. Each team established their own WWW site to disseminate project

information. This created a virtual filing cabinet allowing team members access to the same project information. The focus was not on WWW page prettiness but on 100% content and up-to-date information. BT's VV&T unit published weekly reports, monthly reports, test strategies, test specifications, test reports, training material and access to test systems on the Bali VV&T WWW pages [4].

Fault reporting was conducted using a simple rolling test log in text format converted to Hyper Text Mark-up Language (HTML) by enclosure within the pre-formatted <PRE> and </PRE> tags. Each item was uniquely numbered and its status updated after repeat test cycles. Different project planning tools were used across BT divisions. VV&T project manager workbench (PMW) plans showing a GANTT (a chart with time along the horizontal axis used to display the status of multiple tasks) view were converted into graphics interchange format (GIF) format and posted on to the VV&T WWW site at regular occurrences to eliminate any need for plan duplication in Microsoft (MS) Project.

An initial project meeting was held with all the players meeting face-to-face. Thereafter, all communication was electronic and teleconference with team members remaining within their own locality. Full use was made of BT intranet and teleconferencing services.

2.2 Verification and validation (V&V)

Project V&V was performed through code reviews of CGI C and Oracle PL/SQL code from BT's Belfast Engineering Centre (BEC). Testability analyses on re-

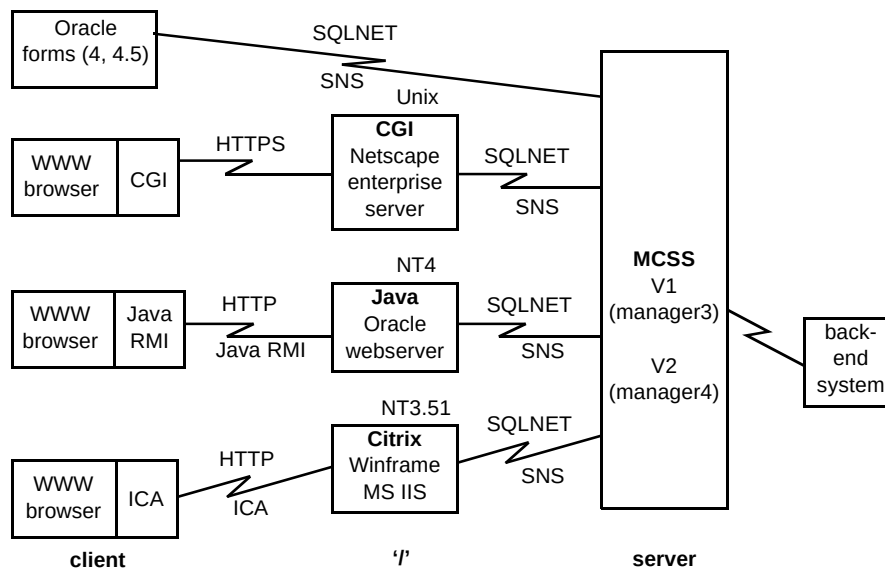


Fig 1 MCSS client/server technologies.

quirement and functional specifications were conducted. This technique aimed to identify problems through static checks which would manifest themselves at a later stage in the software development cycle. A practical approach to conducting a testability analysis was adopted as follows, which is fully documented elsewhere [5]:

- list all statements in the specification by their reference,
- state 'Yes' or 'No' as to whether the statement can be tested,
- for each 'No', include a comment which if answered in the specification would turn it into a 'Yes',
- ignore obvious untestable statements such as document format and wrapping.

An example template is presented in Table 1.

Table 1 Testability analysis template.

Reference	Testable (Yes/No)	Comment
1.0	—	For information only.
2.0	Yes	
2.1	No	Reason — what needs to be added or amended to the statement to make it testable
2.2	No	Reason — what needs to be added or amended to the statement to make it testable
...		

Upon completion, calculate:

$$(\sum \text{Yes} / (\sum \text{Yes} + \sum \text{No})) \times 100\%$$

This provided a metric of a specification's testability and was used as a comparison for other such specifications to improve and tighten specifications. The testability analysis was used to effectively formalise the quality assurance (QA) review process of requirement and functional specifications. The target testability analysis was 100%, with Bali reaching the upper quartile.

2.3 Desktop clients

The Java marketing slogan 'Write once, run anywhere' could be rewritten from a testing perspective as 'Write once, run anywhere, test where?'

The aim was to run the application over the WWW from any desktop, but each desktop and WWW browser had slightly different behaviour and characteristics. The approach taken was to draw up a table (see Table 2) and run a subset of tests for each applicable cell, with full test runs conducted for Netscape Navigator V3.

The WWW browser has security options for turning Java and JavaScript off. This introduced a third dimension to the test matrix since JavaScript was used to validate input through the CGI forms approach. Therefore, turning off data checking at the client end (something you could never do with the Oracle Forms version of MCSS) had the potential to corrupt data on the MCSS database and violate any integrity constraints in operation, such as non-blank fields. At the time, frames were unique to Netscape Navigator and therefore non-frame WWW browsers were tested to ensure the application gave sufficient user warning. The HTML source was viewed from the WWW browser for any user details. Initially user password details were included as parameters to the Oracle Forms executable which appeared as Java applet parameters within the <APPLET> tag.

Table 2 WWW browser and platform test matrix.

Platform	WWW Browser						
	Internet Explorer V2	Internet Explorer V3	Internet Explorer V4 (Beta)	Netscape Navigator V2	Netscape Navigator V3	Netscape Communicator V4 (Beta)	Non-Frames/ JavaScript Browser
Windows 3.11							
Windows 95							
Windows NT3.51							
Windows NT4 Workstation							
Unix X display							

Laptop displays were simulated by changing the display screen size of a desktop PC to 640 × 480 pixels. This proved cost effective and enabled the application to be tested as end users running the application from a laptop would see it.

When testing the Java application, no WWW browser supported the Java Development Kit (JDK) being used, therefore testing was only possible using the appletviewer test harness provided with the JDK. This was run from command lines on both Unix and Windows platforms.

Testing was conducted with secure sockets layer (SSL) enabled on the WWW server to represent how the servers would be run in production. SSL certificates were allocated for a one-year duration unique to each mid-tier server and issued by VeriSign certification authority (CA). With SSL enabled, traffic passed between client and mid-tier server was encrypted.

2.4 Quality

A quality plan was produced for the Bali team's operation. VV&T operated two quality gate checks — one for software from BEC entering into VV&T and one for the final system leaving VV&T for deployment.

Configuration management was local to development and to VV&T, with the configuration management unit (CMU) being the final repository for project material. Bali systems were delivered to VV&T for test via file transfer protocol of a single compressed tape archive file.

The simplified quality approach taken by VV&T on Bali can be summarised as:

- say what you do,
- do it,
- say what you did.

2.5 Test automation

A decision was taken from the project outset to use test automation. This approach followed on from that successfully used on MCSS for Cambridge [6]. Mercury Interactive's WinRunner test tool was used, V3.5 for Windows 3.11 and V4.0.2 for Windows95/NT. A low-level approach was needed since WinRunner synchronised on window screen placement and, using a WWW browser, there was only ever one window on display from a WinRunner perspective. The approach was therefore to provide a function library of WinRunner Test Script Language (TSL) routines which could then be used in test scripts by testers. This had the advantage of hiding low-level TSL (C-like in code syntax) in an application programming interface (API). Test scripts making calls to this API then read more legibly in English. A WinRunner function specification was produced and a development sub-project set up to prepare the set of 20 functions. These were coded and unit tested by VV&T. A test script calling these low-level functions looked like the following:

```
# Test n - login/logout stress test
Bali_NetscapeLaunch ("3"); # launch Netscape V3 browser
Bali_NetscapeOpenLocation ("https://www.sws.bt.com"); # retrieve the login start page
for (i=1; i<=1000; i++)
{
    Bali_Login ("user", "password"); # login to mcss home page
    Bali_Logout (); # logout of mcss
    report_msg ("Login/logout successful for the " & i & "th iteration at time " & time_str);
}
Bali_NetscapeExit (); # close the browser
# End of login/logout stress test
```

The WinRunner generated test report was supplemented by using the report_msg () function to generate a self-contained automated test report upon completion of the test run. The WinRunner TSL file was built to be an all encompassed test specification with no equivalent required in the usual word processed format.

Each test was put into a loop construct initially set to 1. This could then be changed easily to add a stress component

to any test, i.e. fast forward in time to simulate three months of user sessions to see what happens to the server. This proved immensely useful revealing a longevity memory leakage bug in one of the database session processes, which over time would consume system resource and eventually halt the mid-tier server.

WinRunner was used for testing all three technology types — CGI, Java and Citrix WinFrame. However, a new capture/replay test tool called JavaStar was uncovered by the author at a Java testing seminar in Boston, USA which tested Java applications at the Java Virtual Machine (JVM) level. This meant test scripts could be developed once and run on any JVM platform. WinRunner only ran on Windows although it had a sister product called XRunner which ran under Unix. Use of JavaStar was attempted on the Oracle WebForms system but could not be used because of JDK version number and 100% Pure Java issues.

Initial feedback on JavaStar suggests it has massive potential in the VV&T environment once the JDK stabilises and the JavaStar test tool itself matures. JavaStar generates test scripts in Java, performs window verification at the attribute level and provides a graphic editor for test module composition.

2.6 Stress testing

The use of WinRunner allowed a number of 'what if' stress tests. Multiple user testing was done without using any additional tools. The virtual team co-ordinated themselves and arranged a test session where 40 WWW browsers were run simultaneously. Both Unix and Windows desktops were used. Each desktop could run at least 10 separate WWW browsers and therefore initiate the same number of authenticated user sessions. Running 10 separate Oracle Forms sessions would require the same number of physical PCs unless a tool was purchased to simulate an equivalent traffic profile. This technique proved cost effective in assessing how many clients could simultaneously access the system through WWW browsers.

Since WWW browsers are guaranteed to be available on every desktop in the VV&T area, future incentive-based testing could be arranged to facilitate multi-user testing; 20 or so users could be co-ordinated via teleconference to run a series of actions through a WWW browser for an hour. There would be nothing to install locally, not even a uniform resource locator (URL) to type into a WWW browser, but a link to follow from an easily accessible WWW project page. Separate user accounts could be set up on the database servers. All those volunteering to give up an hour over a lunchtime session are bought food and drink as a reward. Compare this to an equivalent load simulator and

its set-up and the cost saving runs into at least 2 orders of magnitude for 80% of the return.

Another approach used was to invoke the Hyper Text Transfer Protocol (HTTP) at the command line through telnet over the HTTP port and to use GET <file> HTTP/1.0 to repeatedly request files from the mid-tier WWW server.

2.7 Test environment

VV&T set up and ran their own test environment. This consisted of a number of MCSS servers of different versions. The mid-tier servers were Unix and Windows NT (versions 4 and 3.51). The clients ranged from Unix (Sun) to all MS Windows types (running on 386, 486 and Pentium processors). A Wyse Winterm Network Computer (NC) was also used for the Citrix WinFrame testing.

All servers used were built and configured by a team independent of development. This activity formed a crucial first phase of testing where installation material was checked and the application analysed to find out whether it could run outside the development environment. The servers were run with logging enabled to extract as much information as possible for problem reports.

All testing in VV&T was done on a local area network (LAN) connected to the BT multi-protocol router network (MPRN) except for dial-up public switched telephone network (PSTN) access testing which used the deployment environment. The latter was done outside normal working hours and proved cost effective, avoiding replicating a costly environment of firewalls and routers.

2.8 Traffic analysis

The new three-tier client/server models being introduced result in an additional server in the equation. A key test activity was assessing just how much extra data traffic, both bytes and packets was introduced as a result. The approach used was to run a Sun Solaris Unix tool called 'snoop' to capture and inspect network packets. Root access to a Unix machine on the same LAN segment was required to capture packets specifically between the client and mid-tier, and between the mid-tier and the MCSS database server (see Fig 2). These measurements were then compared to similar measurements taken from the traditional client/server system. A set of 12 tests featuring functionality areas such as log-in, log-out, create record, list records were used and these were compared between the different three-tier model technologies.

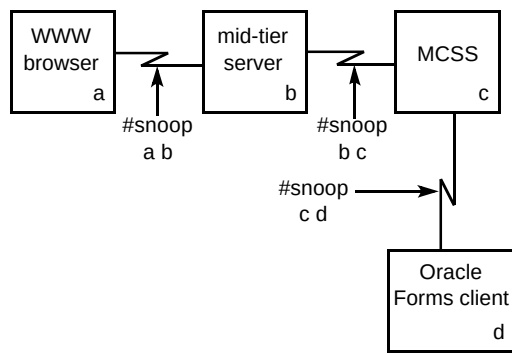


Fig 2 Bali traffic analysis approach.

The log-in area proved to be the most traffic intensive compared to its equivalent function in an Oracle Forms environment. For the Java application, the initial applet download time caused this increase in traffic, which was reduced somewhat by using Java Archive (JAR) compression within the <APPLET> tag. For Citrix WinFrame, the log-in activity included session establishment with the Citrix NT server. For each horizontal/vertical toggle of the MCSS floating control panel, approximately 5000 bytes were passed between client and mid-tier server over a LAN. The equivalent traffic for the traditional Oracle Forms client/server system was 0.

The raw snoop files were examined through VV&T-built Unix awk (pattern scanning and processing language) scripts to count both packets and bytes for each traffic test.

The CGI approach introduced the least data traffic increase to the standard Oracle Forms system and the Java applet approach introduced the largest increase in data traffic.

3. The WWW technologies investigated

3.1 CGI

The CGI approach involved a series of CGI executables running on a mid-tier Unix Solaris V2.5 server with Netscape Enterprise Server (NES) V2 and Oracle V7.3. The interface was to MCSS (Manager V3), (Unix Solaris V2/ Oracle V7) which authenticated user sessions. The system provided the ability for customers to create requests from a WWW browser which raised transactions into the MCSS database. A simple push like red/green alert frame indicated to a user a notification of change in status to their request, red indicating change. A BEC-invented URL hash encoding scrambling mechanism ensured the security of the user session and eliminated the need for any re-authentication upon page movement within the WWW browser (Netscape or Internet Explorer).

The system introduced the concept of a shared workspace (SWS) component where BT and its customers

could share information within a closed environment (extranet). This functionality spawned customised versions of DEC Alta Vista Forum V2.0 and SWS releases which also came through VV&T for testing.

The disadvantage with the CGI approach was that CGI processes were created and removed as required which affected performance. The application logic was duplicated on to the mid-tier server in addition to being on the MCSS database server which increased any maintenance effort. Key test areas were server scalability and checking that the duplicated MCSS application logic on the mid-tier server did not affect operations for clients using native Oracle Forms.

3.2 Oracle WebForms

The Oracle WebForms approach involved using the Oracle Developer 2000 Web Cartridge V1.4w to run the existing Oracle Forms V4.5 MCSS Manager V4 application as a Java applet through Java Remote Method Invocation (RMI) introduced in JDK V1.1. The mid-tier server ran Oracle Web Server V2.1 on a Windows NT4 Server platform which interfaced to MCSS (Unix Solaris V2/ Oracle V7). A number of Beta versions were taken from Oracle and the Java applets were run through an Oracle-provided appletviewer test harness on Windows95/NT clients. The application would then run in an identical manner to the standard Oracle Forms V4.5 version on a PC with no changes required to the application other than porting to 32-bit windows mode. BT's Glasgow Engineering Centre performed the 32-bit Windows port and provided the initial installation material. The release then came into VV&T for testing.

Various difficulties existed with third-party products which meant progress was not as expected. A significant issue was the rate of change with the JDK at the time. The key test areas were help files, print options, login and mid-tier server process footprint sizing to identify optimum mid-tier server configuration. Fault reporting was categorised into four areas — forms, user interface, administration and resource. Further, many defects were raised against third-party products with a relatively small proportion against MCSS itself.

3.3 Citrix WinFrame

The Citrix WinFrame approach involved running the MCSS Manager V3 Oracle Forms V4 application from a mid-tier server running Citrix WinFrame V1.6 (service pack 5) on Windows NT3.51 with MS Internet Information Server (IIS) V1.0 WWW server. The application ran unchanged on the mid-tier with graphics device interface (GDI) calls intercepted by WinFrame and redirected through its independent computing architecture (ICA) protocol to the thin client. The client can therefore run the

compute-intensive MCSS application from any PC capable of running MS Windows and a WWW browser (i.e. 386 PC). The traffic between the client and mid-tier was screen content, keyboard entries and mouse movements. The application was launched from a Netscape browser (with the Citrix plug-in installed) or Internet Explorer. The application was also launched through a Wyse Winterm NC which had the ICA protocol built into its hardware.

The key test areas were identifying conflicts in shared resources (initialisation files, log files, configuration files), print options, secure sessions and identifying application footprint sizes on the mid-tier server. VV&T also led the activity to identify optimum Citrix WinFrame configurations based on numbers of concurrent user sessions. The mid-tier server requirement was potentially 4 × CPUs with 1 Gigabyte of RAM running on Windows NT3.51 depending upon how many concurrent users were required. A difficulty found with this approach was trying to establish the user process footprint size on the mid-tier server with the available NT tools and without having access to the appropriate high specification NT hardware. An additional 25% memory and 20% processor overhead was incurred due to running the MCSS application as a 16-bit application within a 32-bit environment. A system administration monitor shareware tool was adopted to aid the process, similar to NT4 task manager.

Key to the project was the need for a Citrix-specific set of requirements which identified clearly the configuration options to be made by the Citrix administration tools. In addition, traffic analysis would reveal the optimum placement of the mid-tier server, i.e. close to the clients or close to the MCSS server, and any impact to the deployment LAN infrastructure.

This activity was integration-led due to the components being available off the shelf. No changes were required to the MCSS application using Citrix WinFrame.

4. The future

When the JDK becomes stable and WWW browsers and NCs offer support for that version, then Java will become a widespread delivery mechanism. Java specific test tools will become widely available. SunTest's suite of JavaStar capture/replay tool, JavaSpec for unit test and JavaScope for code coverage are a good start. Windows NT will increasingly become the mid-tier platform of choice and various administration tools will be available to measure load and performance as exists on Unix today.

All the MCSS intranet work to date has been within the protection of the BT firewall. It is inevitable that full Internet access will be required to facilitate a wider customer reach. With this comes the additional testing for security and volume user management.

The work was conducted at BT Laboratories in the UK. The next step will be to perform the same type of work with VV&T resource offshore to take advantage of both cost and time-zone differences. Servers would be set up in the UK by close of play UK time and be ready for unit test/system testing offshore. This would lead to reduced time-scales and provide a way of running VV&T within 'Internet time-scales'. The only hardware/software requirements for offshore being a desktop device capable of running WWW browser-launched applications and a BT MPRN connection.

5. Conclusions

The lessons learnt on WWW enabling MCSS are that VV&T can and must be done on Internet WWW applications. The temptation to rely on end users testing WWW applications, omitting the VV&T function to save time, has proved not to work. Equally, a rigid and formal VV&T function has been found inappropriate for a WWW project. A new approach, capable of allowing VV&T to operate on 'Internet time' was found to be the most beneficial to the project:

- produce a simplified test strategy which is, effectively, say what you do, do it and say what you did,
- pick and mix from tried and trusted VV&T techniques,
- the Art of Software Testing [7] never ages and remains the definitive source on testing principles,
- try new things and innovate constantly,
- use the 80/20 rule,
- use the WWW for all project and test material — other parts of the business will certainly be thinking along the lines of WWW enabling their operational support systems [8] and any documentation produced which is available to all in a convenient manner will help eliminate 'reinventing the wheel',
- adopt test automation from the start but keep it simple,
- try lots of 'what if' tests and use the function library approach to hide the tool internals where necessary,
- use hybrid tailored RAD/DSDM techniques,
- look at SI-Rapid,
- adopt and trust virtual teams to deliver,
- become familiar with Internet technology — understand what it is, what it can do and what the current Internet trends are; from this will come an understanding of how to test Internet and WWW computer systems and what new areas need to be addressed in the three-tier thin-client/server systems now being developed.

Acknowledgements

The author acknowledges all of the Bali MCSS intranet team members past and present for their work and vision on the project. Thanks also to Pavel Kricka, Richard Tinker and Bruce Boxall for reviewing this paper.

References

- 1 White P: 'A service system for BT's major customers', BT Technol J, 15, No 1, pp 81—86 (January 1997).
- 2 Carter C J et al: 'Rapid integration and testing of business solutions', BT Technol J, 15, No 3, pp 37—47 (July 1997).
- 3 BT: 'VV&T Handbook, Issue 3', Internal BT document (1996).
- 4 BT: 'http://www.airport.bt.co.uk/sect_hand/gmcs/bali/bali.htm' (internal BT Web page).
- 5 BT: 'Systems VV&T News, Issue 12', Internal BT document (September 1995).
- 6 Hicks I D et al: 'Automated testing as an aid to systems integration', BT Technol J, 15, No 3, pp 26—36 (July 1997).
- 7 Myers G: 'The Art of Software Testing', Wiley (1979).
- 8 BT: 'Browsing the COSMOSS', Competitive Edge, Issue 12, (internal BT publication) (July/August 1997).



Russell Croucher joined BT in 1988 after five years as a computer programmer/systems analyst at Sema Group in London. He graduated with a BSc honours degree in Mathematics and Computer Science at Sheffield University in 1983. His work for BT has covered software development and VV&T. In 1988, he worked on developing a call logging system called Linesman Logging Manager. Since 1989, he has worked in all stages of the software development life cycle of the BT ServiceView/MCSS network and service management programmes, concentrating in the latter years as a team leader in VV&T activities. He specialises in software development test and automation tools.